

姚顺雨的 ReAct 与 SWE-agent

—— 用计算机系同学能听懂的话，讲清楚 AI 智能体 (Agent)

从「推理 + 行动」的协同，到「为智能体设计接口」

整理 · Z.ai 2026-06-24

目录

- 一、先认识作者：姚顺雨是谁
- 二、几个前置概念（CS 视角的快速对齐）
- 三、ReAct：让模型「边想边做」
- 四、SWE-agent：把循环用到「改代码」这件事上
- 五、把两个概念串起来：对比总结
- 六、参考资料来源

一、先认识作者：姚顺雨是谁

在讲概念前，先简单交代一下作者背景，方便你理解他为什么做这些研究。

教育经历

- 清华姚班本科（2015 - 2019），普林斯顿大学计算机系博士（2019 - 2024），导师是 Karthik Narasimhan 教授。
- 博士论文题目是《Language Agents: From Next-Token Prediction to Digital Automation》（语言智能体：从「预测下一个词」到「数字自动化」）——这基本概括了他整个博士期间的研究主线。

代表成果与职业经历

- 代表作：ReAct（ICLR 2023，引用量过万）、Tree of Thoughts（思维树）、SWE-bench / SWE-agent、CoALA（语言智能体概念框架）。
- 2024 年加入 OpenAI，参与 Computer-Using Agent、Deep Research 等项目；2025 年发表引发热议的《The Second Half》（AI 下半场）博文；现已回国加入腾讯任首席 AI 科学家。

他研究的核心问题其实就一句话：怎么让大语言模型（LLM）不再只是「聊天机器人」，而变成能动手干活的「智能体（Agent）」。ReAct 和 SWE-agent 正是这条线上的两个里程碑。

二、几个前置概念（CS 视角的快速对齐）

讲 ReAct 之前，先把几个词对齐一下，因为你可能没跟过 NLP 这几年的进展。我用你熟悉的类比来解释：

概念	你熟悉的类比
LLM（大语言模型）	一个超级强大的函数 $f(\text{text}) \rightarrow \text{text}$ ，输入一段话、输出一段话。本质上是「给定前文，预测下一个词」。
Prompt（提示）	给这个函数的输入参数。
Chain-of-Thought（思维链 CoT）	让模型「把推导过程写出来」再给答案，类似你在草稿纸上一步步算，而不是心算，能显著提高正确率。
Tool use / Acting（工具使用 / 行动）	让模型不只是输出文字，还能调用外部工具（搜索、查数据库、运行代码），拿到结果再继续。
Agent（智能体）	一个以 LLM 为「大脑」、能感知环境、做决策、执行动作、再根据反馈调整的循环系统。可理解为「以 LLM 为控制流的程序」。

表 1 AI Agent 领域核心概念与 CS 类比对齐

关键张力在于：在 2022 年之前，「推理（reasoning，比如 CoT）」和「行动（acting，比如调工具）」是两条分开的研究路线。ReAct 的贡献就是把它们缝合在一起。

三、ReAct：让模型「边想边做」

论文信息

标题	ReAct: Synergizing Reasoning and Acting in Language Models
作者	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao
发表	ICLR 2023 (arXiv: 2210.03629, 2022 年 10 月首发)

它要解决什么问题

想象你用 LLM 回答一个问题：「清华大学 2018 年的校长是谁？」

- 纯推理（CoT）路线：模型只能靠记忆里「挤」答案。训练数据里有就答对；没有或记错了，就会一本正经地胡说八道（幻觉，hallucination）。它没法去查证。
- 纯行动路线：模型直接调搜索引擎拿到一堆网页，但它不会「想一想该查什么、查到的够不够、要不要换关键词」，容易陷入无效操作，错误还会一路传播。

ReAct 的洞察是：这两件事天生该配合着做。你查资料的时候，不也是「想一想 → 查一下 → 看结果 → 再想想 → 再查」这样交替进行的吗？

核心机制：Thought-Action-Observation 循环

ReAct 让模型按固定格式，交替产生三类内容：

```
Thought[REDACTED]: [REDACTED].....  
Action[REDACTED]: Search["[REDACTED] 2018"]  
Observation[REDACTED]: <[REDACTED]>  
Thought: [REDACTED] 2015-2023 [REDACTED].....  
Action: Lookup["[REDACTED]"]  
Observation: <[REDACTED]>  
Thought: [REDACTED]2018 [REDACTED]  
Action: Finish["[REDACTED]"]
```

用计算机系的话翻译这三类内容：

- Thought = 模型的内部「日志 / 调试输出」，即推理过程（reasoning trace）。它不被外部执行，只是帮模型自己理清思路，也帮人类看懂它为什么这么做。
- Action = 一次对外部环境 / 工具的函数调用（搜索、查表、跳转、Finish 等），有严格格式。
- Observation = 工具返回的结果，作为下一轮的输入。
- 这三者构成一个循环（agent loop），直到模型发出 Finish 动作为止。

这本质上是把 LLM 当成一个解释器在跑的、用自然语言写的控制程序：Thought 是注释和条件判断，Action 是函数调用，Observation 是返回值。

为什么这个设计有效

论文在 HotpotQA 多跳问答、ALFWorld 文字游戏、WebShop 网购模拟等任务上做了实验，发现：

- 协同 > 单干: ReAct 比纯 CoT、纯 Acting 都好。推理帮它规划下一步该干嘛，行动帮它拿到真实信息来支撑推理。
- 减少幻觉: 关键事实从外部工具拿到，不是模型瞎编的。
- 可解释性更好: Thought 是白盒的，人能看见模型的「思考过程」，方便调试。
- 也有个权衡: 纯 CoT 在模型「本身就会」的任务上推理更强，而 ReAct 更适合需要外部知识 / 交互的任务。后来还有人把两者结合（先 ReAct，答不上来再回退到内部推理）。

它的地位

ReAct 的意义在于它确立了一个范式（paradigm）。今天你看到的几乎所有 AI agent 框架——LangChain、AutoGPT、各种「能上网能写代码的 AI 助手」——底层几乎都是 ReAct 这个「思考-行动-观察」循环。它把 agent 从概念变成了一个可复用的工程模板。作者在 GitHub 上也建议：实际用就直接上 LangChain 的 ReAct 实现，他的仓库只是教学用的最小实现。

四、SWE-agent：把循环用到「改代码」这件事上

理解了 ReAct，SWE-agent 就好懂了——它是 ReAct 思想在「软件工程」这个垂直领域的强化版。

论文信息

标题	SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering
作者	John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, Ofir Press（普林斯顿 NLP 组）
发表	NeurIPS 2024 (arXiv: 2405.15793, 2024 年 5 月)

注：姚顺雨是共同作者之一，整个项目出自他所在的普林斯顿实验室。它和姊妹工作 SWE-bench 配套：SWE-bench 是「考题」（从真实 GitHub 仓库收集的 issue + 测试用例，用来考 AI 修 bug 的能力），SWE-agent 则是去「考试的考生」。

它要解决什么问题

给 agent 一个真实软件仓库里的 GitHub issue，比如：

「运行 `python main.py --debug` 时报 `KeyError: 'config'`，期望它能优雅降级。」

让 agent 自主完成人类工程师会做的事：浏览仓库文件、找到相关代码、读懂逻辑、修改、跑测试、改对。这比 ReAct 论文里的「调一下搜索」难多了——它要在一个完整的代码仓库 + 终端里操作。

直接把 ReAct 套上去效果不好。作者发现，瓶颈不在模型本身，而在「模型怎么和计算机交互」这件事上。

核心创新：ACI（Agent-Computer Interface，智能体-计算机接口）

这是 SWE-agent 最重要的概念，论文标题里的「Agent-Computer Interfaces」就是它。

类比一下：人类程序员写代码靠的是 IDE / 人机接口（HCI）——有文件树、语法高亮、跳转定义、错误提示、git diff，这些 HCI 设计得好，人写得又快又准。

SWE-agent 说：agent 也需要一套专门为它设计的「人机接口」，只不过对面不是人，是 LLM，所以叫 ACI。现成的 shell 和编辑器是给人用的，对 LLM 不友好（输出太杂、反馈格式不固定、容易让模型迷糊）。于是他们专门设计了一套 ACI，给 agent 提供几个格式干净、反馈明确的工具命令，比如：

- `find_file / search_dir / search_file`：在仓库里找文件和代码，返回结果规整成容易「读」的格式；
- `open / goto / scroll`：浏览文件，每次只显示一个固定大小的「窗口」（就像人一次只看一屏），避免模型被几千行代码淹没；
- `edit`：编辑代码，并强制打开修改后的文件让模型核对，防止「改完就忘」；
- `python`：运行代码 / 测试，把终端输出处理干净再喂回去。

每个工具的输入输出都被精心设计成「对 LLM 友好」的形式。设计原则包括：

- 简洁、明确：输出要紧凑、格式固定；
- 窗口化、可操作：信息要分块，agent 能逐块操作（像人滚动屏幕）；
- 反馈要让人（模型）「看见后果」：任何改动之后，立刻把结果反馈给模型确认。

这套设计的思路其实非常「工程化」：你不是在等一个更聪明的模型出现，而是在改「API 的设计」——就像给一个新来的实习生设计一套好用的内部工具和规范，他就能干得更好。作者甚至发现，用更弱的模型 + 更好的 ACI，能打过更强的模型 + 差的接口。

效果

在 SWE-bench 上，SWE-agent 当时拿到了 SOTA（最先进水平），能自主解决相当比例的真实 GitHub issue。更重要的是它证明了一个论点：做 agent，除了堆模型能力，「接口设计」是被严重低估的杠杆。

五、把两个概念串起来：对比总结

	ReAct	SWE-agent
解决的问题	怎么让 LLM 边推理边行动（通用范式）	怎么让 LLM 自主完成软件工程任务（垂直应用）
核心机制	Thought - Action - Observation 循环	在 ReAct 式循环之上，重点设计了 ACI（接口）
关键贡献	提出并验证了「推理 + 行动协同」的范式	提出「为 agent 设计专属接口」这个被忽视的维度
地位	agent 领域的奠基性范式	agent 在软件工程领域的里程碑 + 工程方法论
关系	提供「骨架」（循环结构）	在骨架上针对「操作代码仓库」做深度优化

表 2 ReAct 与 SWE-agent 对比

一句话总结这条线索：

ReAct 回答了「agent 该怎么想、怎么做」（建立循环范式）；SWE-agent 回答了「当 agent 要在一个复杂环境（代码仓库 / 终端）里做事时，怎么给它设计一套好用的交互接口，让它干得又准又稳」。

姚顺雨的研究主线就是把这些一步步做实——从「模型能推理」到「模型能行动」，再到「模型能在真实的数字世界里自动化地完成专业工作」。这也是为什么他的博士论文标题是「从预测下一个词，到数字自动化」。

六、参考资料来源

ReAct

- [ReAct 论文 \(arXiv:2210.03629\)](#)
- [ReAct 项目主页](#)
- [ReAct ICLR 2023 论文 PDF](#)
- [作者代码仓库 vsymyth/ReAct](#)
- [Google Research Blog 对 ReAct 的解读](#)
- [Latent Space 对姚顺雨的访谈](#)

SWE-agent / SWE-bench

- [SWE-agent 论文 \(arXiv:2405.15793\)](#)
- [SWE-agent 官方文档与架构介绍](#)
- [SWE-agent ACI 设计说明](#)
- [NeurIPS 2024 正式版 PDF](#)

作者背景

- [姚顺雨个人主页](#)
- [Google Scholar 主页](#)